INSTITUTO JUAN AMÓS COMENIO

Analista de Sistemas · Proyecto Final

PSIQUE

Seguimiento psicoterapéutico
y administración de consultorio

Documentación técnica

AUTOR

Franco José Caudeli

DOCENTE

Julián Speciale

Edición académica · 1 de noviembre de 2026

Informe técnico · Versión 1.0

Contenido

Lectura orientada a la versión comprobada al 01/11/2026. Las capturas contienen datos ficticios y los estados pendientes se identifican expresamente.

1. Descripción técnica y estado verificado 3
2. Arquitectura de tres capas 4
3. Tecnologías y versiones utilizadas 5
4. Organización del repositorio 6
5. Frontend Ionic y Angular 7
6. Flujo de autenticación y navegación 8
7. Backend y ciclo de vida del servidor 9
8. API pública y autenticación 10
9. API privada de consultorio 11
10. Contratos JSON, fechas y errores HTTP 12
11. Modelo entidad-relación 13
12. Diccionario de datos: usuarios 14
13. Diccionario de datos: pacientes 15
14. Diccionario de datos: turnos 16
15. Diccionario de datos: sesiones 17
16. Integridad, tipos e índices 18
17. Migraciones y administración del esquema 19
18. Transacciones y control de concurrencia 20
19. Autorización, JWT y seguridad HTTP 21
20. Integración de Resend y enlaces de cuenta 22
21. Android con Capacitor y conectividad 23
22. Variables de entorno y secretos 24
23. Instalación nueva en Windows y PostgreSQL 25
24. Ejecutar web, API y pruebas localmente 26
25. Compilar, instalar y ejecutar Android 27
26. Pruebas ejecutadas y evidencia 28
27. Despliegue previsto y resguardos pendientes 29
28. Operación, trazabilidad y referencias 30

1. Descripción técnica y estado verificado

PSIQUE utiliza una arquitectura cliente-servidor con tres capas: presentación, aplicación y persistencia. Ionic y Angular implementan la interfaz. Node.js y Express aplican validaciones, autenticación y reglas de negocio. PostgreSQL conserva cuentas, pacientes, turnos y sesiones.

El frontend se ejecuta como aplicación web y también se empaqueta en Android mediante Capacitor. Ambas variantes utilizan la misma API REST. La versión Android no ejecuta Express ni PostgreSQL dentro del dispositivo y no incorpora un almacenamiento clínico sin conexión.

Línea de base de esta entrega

La revisión corresponde a la copia de trabajo del **1 de noviembre de 2026**, incluidos los archivos de integración Android aún sin commit. Las versiones de dependencias se obtuvieron de los package-lock.json y del entorno instalado. No se presenta el último commit como si incluyera esos cambios pendientes.

PostgreSQL es el único motor del servidor habitual. El código, archivos y pruebas SQLite permanecen como antecedentes; no hay fallback ni doble escritura. No se importaron registros desde SQLite.

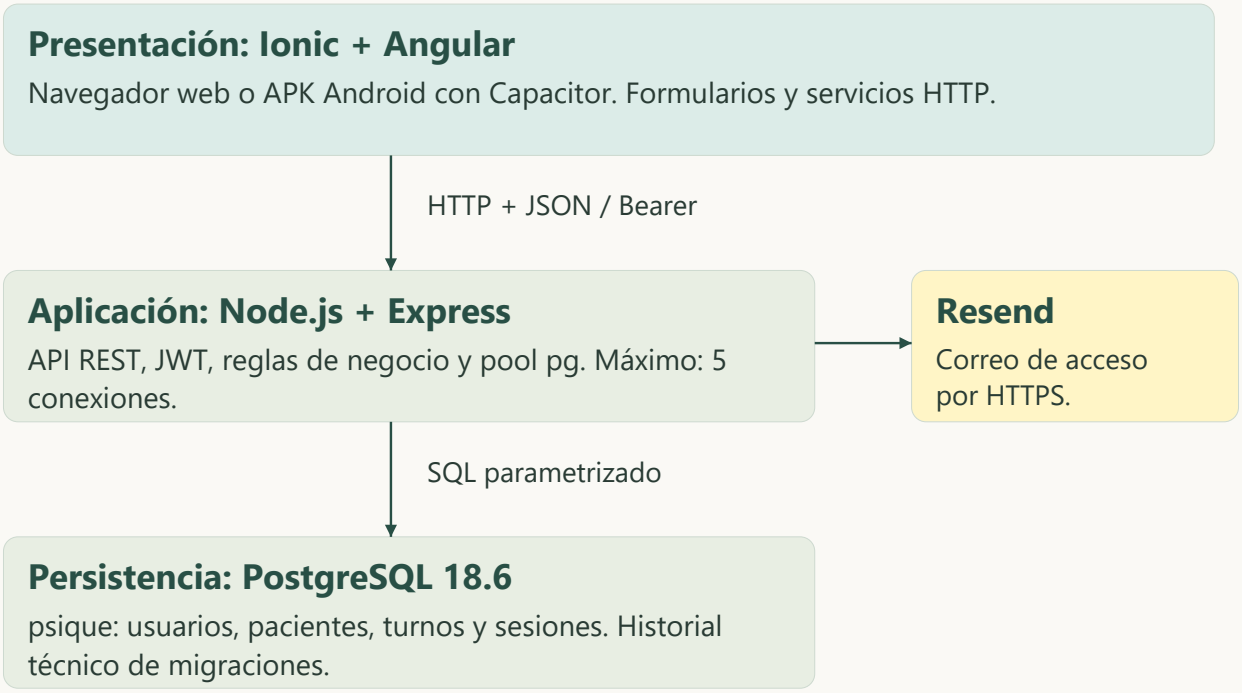
Qué se comprobó

Se reejecutaron las suites actuales, la compilación web y la verificación de firma del APK. Se inspeccionaron código, migraciones, configuración y documentación operativa. Las capturas se tomaron desde el APK instalado contra una API aislada en psique_test. Los archivos de evidencia acompañan esta entrega.

La consigna original no se encontró como archivo independiente. Se revisó el Documento Funcional de octubre como antecedente y se utilizó el pedido formal de documentación como requisito de entrega. Las descripciones actuales se contrastaron con el código cuando el antecedente o el README conservaban información histórica.

Este documento explica instalación y operación local reproducibles sin distribuir el almacén privado de credenciales de la computadora del autor.

2. Arquitectura de tres capas



Capa	Responsabilidad real
Presentación	Componentes Ionic/Angular, navegación, formularios, estados de carga y servicios HttpClient.
Aplicación	API Express, middleware JWT, validadores, calendario, reglas de cobro y transacciones SQL.
Persistencia	PostgreSQL 18.6, tablas relacionadas, restricciones, índices y versionado del esquema.

El cliente envía HTTP con JSON. Las operaciones privadas incluyen Bearer. Express obtiene la identidad validada, ejecuta SQL parametrizado mediante pg y responde JSON. En escrituras de negocio espera COMMIT antes de informar éxito.

Resend es una dependencia externa de envío de correos por HTTPS. No recibe pacientes, sesiones ni contenido clínico desde el adaptador de acceso: recibe destinatario, asunto y texto del enlace de cuenta.

La ejecución comprobada es local. Web utiliza 127.0.0.1:4200 y API 127.0.0.1:3000. PostgreSQL escucha en el puerto 5432. Android llega a la API local mediante ADB reverse. El diagrama no representa un despliegue en la nube.

Fuente: src/app/services/, backend/src/server.ts, app-postgres.ts, api.ts y db/consultorio-postgres.ts.

3. Tecnologías y versiones utilizadas

Componente	Versión comprobada	Función
Node.js / npm	24.14.1 / 11.11.0	Runtime y gestión de dependencias.
Angular	21.2.22	Componentes, rutas, formularios y HTTP.
Angular CLI / Build	21.2.23	Servidor de desarrollo y compilación.
Ionic Angular	9.0.2	Contenedores, navegación y componentes de interfaz.
TypeScript	5.9.3	Tipado en frontend y backend.
RxJS	7.8.2	Flujos asíncronos de HTTP y estado de pantalla.
Express	5.2.1	API REST y middleware.
pg	8.23.1	Pool y acceso PostgreSQL sin ORM.
PostgreSQL / psql	18.6	Motor de persistencia y administración local.
argon2 / jose	0.45.1 / 6.2.12	Hash Argon2id y JWT HS256.
Helmet / cors	8.3.0 / 2.8.6	Cabeceras y política de origen HTTP.
express-rate-limit	8.7.1	Límites de solicitudes de autenticación.
Capacitor core / CLI / Android	8.6.0	Contenedor y herramientas Android.
Capacitor App	8.1.2	Integración del botón Atrás con Ionic.
Android Studio	2026.2.2	IDE y gestión del SDK/emulador.
JDK / Gradle / AGP	21.0.13+9 / 8.14.3 / 8.13.0	Compilación del contenedor nativo.
SDK Android	min 24, compile/target 36	Compatibilidad declarada del proyecto.
Emulador probado	Android 17, API 37	Instalación y recorrido reales del APK.

better-sqlite3 13.0.3 permanece para pruebas y código histórico. No lo utiliza el arranque habitual. Resend se consume con fetch de Node, sin SDK adicional.

El entorno es Windows x64. No se requiere Angular CLI o Ionic global. Las versiones declaradas pueden ser rangos; los lockfiles fijan la resolución utilizada. No se acredita compatibilidad ejecutada con todas las versiones Android desde minSdk 24.

4. Organización del repositorio

```
APP-PSIQUE/
  src/app/              Pantallas, modelos, servicios y autenticación Angular
    shared/             Calendario, fechas, importes y estilos comunes
  src/environments/     Configuración pública de destino de la API
  scripts/             Compilación web de producción y preparación Android
  test-config/         Pruebas de configuración y formularios Angular
  android/             Proyecto nativo Gradle/Capacitor
  capacitor.config.ts  Identidad Android y webDir www/browser
  angular.json         Build web, producción y android-local
  backend/
    src/server.ts      Arranque y cierre PostgreSQL
    src/api.ts         Rutas HTTP y contratos
    src/auth*.ts       Argon2id, JWT y almacenamiento de usuarios
    src/cuenta-correo.ts / correo.ts  Flujos y adaptador Resend
    src/db/            Pool y consultas de consultorio PostgreSQL
    migrations/        Versiones SQL 001 a 008
    scripts/           Migrador, comprobaciones y configuración SQL
    test-postgres/     Pruebas actuales con PostgreSQL aislado
    test/              Pruebas históricas y reglas
  www/browser/         Resultado Angular generado, no editable a mano
  entrega_academica/   Documentos, fuentes, imágenes y evidencias
```

La raíz y backend tienen package.json, lockfile y node_modules independientes. backend/dist contiene JavaScript compilado. www/browser contiene HTML, CSS y JavaScript para navegador o para sincronizar al contenedor Android; no contiene la API ni la base PostgreSQL.

Los directorios de dependencias y compilación, APK, configuración personal de SDK y secretos se excluyen del versionado. Los documentos de esta entrega se organizan fuera de los directorios de ejecución.

El código SQLite de app.ts y db/database.ts se mantiene como referencia. server.ts importa la implementación PostgreSQL y no decide el motor según disponibilidad.

5. Frontend Ionic y Angular

Las pantallas son componentes standalone cargados por demanda desde app.routes.ts. La raíz separa las pantallas públicas del contenedor espacio-privado. Este último mantiene ion-router-outlet y la navegación inferior Inicio, Pacientes y Agenda.

Presentación y estado

Reactive Forms define controles, validadores y borradores. Las pantallas indican carga, guardado y errores. Guardar convierte importes a centavos y envía la operación HTTP. Cancelar restaura los datos anteriores. Las solicitudes se cancelan o suscriben según el ciclo de vida mediante Subscription y takeUntilDestroyed.

Ionic puede conservar pantallas en memoria; ionViewWillEnter actualiza datos al volver. Al cerrar sesión se destruye el contenedor privado para no reutilizar fichas o listados de otra cuenta. El modal de cobros también se retira.

Servicios y modelos

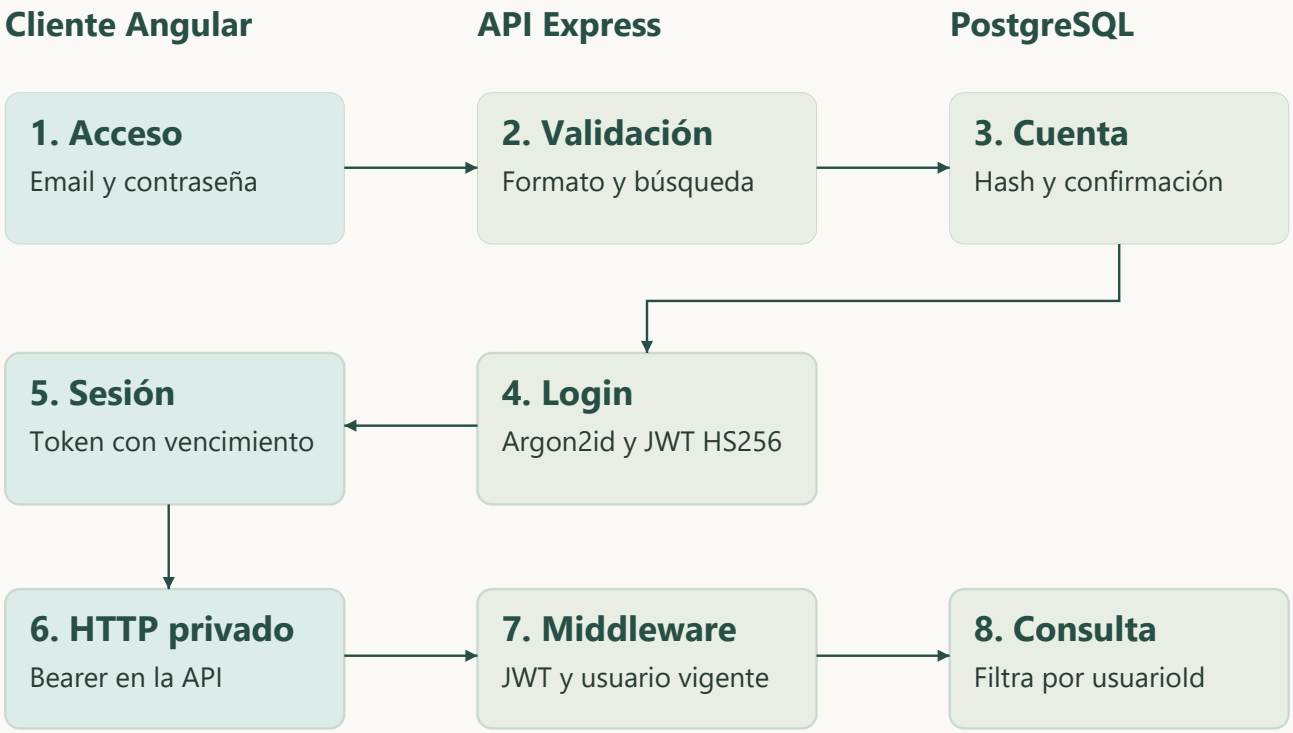
Servicio	Responsabilidad
AuthService	Cuenta, correo, token, vencimiento y logout.
PacientesService	Listar, sugerir código, consultar, crear, editar, suspender y dar de baja.
SesionesService	Calendario, reservas, consultas, histórico, realización, edición y cancelación.
DashboardService	Resumen numérico calculado por la API.
AgendaService	Consulta semanal anterior conservada por compatibilidad.

API_BASE_URL deriva de environment.ts. Producción y android-local usan reemplazos explícitos. La URL del frontend es pública y no contiene credenciales PostgreSQL ni claves de correo.

La paginación de cinco registros en Últimas sesiones y en el histórico individual se hace en Angular sobre el arreglo completo ya ordenado por la API. No existe paginación SQL de esos endpoints; su costo crecerá con la cantidad de registros.

Fuente: src/app/app.routes.ts, services/, models/, shared/, paciente-historico/ y sesiones/.

6. Flujo de autenticación y navegación



El login responde **{ token, usuario, expiraEn: 3600 }**. Usuario contiene solo id, email y fechaCreacion. No devuelve el hash de contraseña ni información clínica.

AuthService conserva token y usuario en memoria y sessionStorage, sin almacenar la contraseña. Lee exp para programar el vencimiento visual. Esta lectura del cliente no comprueba la firma: la autenticación efectiva se realiza en Express.

El interceptor compara origen y ruta con API_BASE_URL. Agrega Authorization: Bearer únicamente a endpoints privados de esa API. Excluye registro, login y los cuatro flujos de correo; tampoco envía el token a servidores externos. Descarta respuestas tardías de una cuenta que ya cerró sesión. Un 401 de la sesión vigente provoca logout y redirección.

authGuard cuida canActivate y canActivateChild. accesoGuard evita mostrar Login/Registro si ya hay sesión local. El guard mejora la navegación y no sustituye la autorización del servidor.

El logout borra la sesión local. No existe un endpoint de logout ni una lista de revocación individual. Un JWT copiado puede seguir válido hasta expirar, salvo cambio de versionCredenciales por restablecimiento de contraseña.

Fuente: backend/src/auth.ts, auth-postgres.ts y src/app/auth/, services/auth.service.ts.

7. Backend y ciclo de vida del servidor

server.ts valida JWT_SECRET y la configuración HTTP y de correo. Después espera abrirPoolPostgres. El pool exige DATABASE_URL con psique_app/psique y comprueba la identidad devuelta por el servidor, la ausencia de privilegios administrativos y el acceso a todas las columnas esperadas.

Solo entonces se crea la API completa y se abre HTTP. Las comprobaciones SELECT ... LIMIT 0 no leen registros clínicos ni crean tablas. Express no ejecuta migraciones.

Conexiones y asincronía

pg administra un pool con máximo predeterminado de **5 conexiones**, espera de conexión de **5 segundos**, cierre por inactividad a los **30 segundos** y límites de consulta/sentencia de **10 segundos**. Las consultas independientes usan pool.query; una transacción toma un cliente exclusivo.

async/await espera las respuestas PostgreSQL antes de continuar. No convierte por sí solo una operación en transacción ni confirma sus cambios. El proceso puede atender otras solicitudes mientras espera la base.

Fallo y cierre

- Si faltan secreto, conexión, permisos o esquema, no se inicia una API parcial.
- Un error de puerto cierra el pool que se haya abierto.
- No se abre SQLite ni se crea otra base como reemplazo.
- En funcionamiento, problemas de conexión se traducen a 503 y errores SQL inesperados a 500 genérico.
- SIGINT/SIGTERM dejan terminar solicitudes y correos en curso y cierran el pool. Tras 10 segundos se fuerzan las conexiones HTTP pendientes. El cierre es idempotente.

/health/live responde sin consultar SQL. /health/ready usa SELECT 1, comparte consulta en curso y resultado por 5 segundos y tiene una espera aproximada de 1,5 segundos. Es un indicador de disponibilidad, no una garantía de éxito de la siguiente escritura.

Fuente: backend/src/server.ts, db/postgres.ts, disponibilidad.ts y app-postgres.ts.

8. API pública y autenticación

La base local es **http://127.0.0.1:3000/api**. Las rutas siguientes no necesitan JWT. Los enlaces de confirmación y recuperación utilizan su propio token de un solo uso.

Método y ruta relativa	Entrada	Respuesta principal
POST /auth/registro	email, password, repetirPassword	201: usuario público y correoEnviado.
POST /auth/login	email, password	200: token, usuario y expiraEn.
POST /auth/reenviar-confirmacion	email	200: mensaje genérico de solicitud.
POST /auth/confirmar-email	token del enlace	200: email confirmado.
POST /auth/recuperar	email	200: mensaje genérico de solicitud.
POST /auth/restablecer	token, password, repetirPassword	200: cambio de contraseña confirmado.

Fuera de /api, GET /health/live y GET /health/ready son públicos y solo informan un estado genérico.

Registro admite exactamente los tres campos indicados. El email se recorta y normaliza a minúsculas. La contraseña no se modifica y en alta/restablecimiento debe tener 12 a 128 caracteres. Un email inexistente y una contraseña equivocada reciben el mismo 401. Una cuenta nueva no confirmada con contraseña correcta recibe 403 sin JWT.

El registro no inicia sesión automáticamente. Si falta configurar el proveedor de correo, se rechaza antes de crear la cuenta con 503. Si el envío falla después del alta, responde 201 con correoEnviado=false y permite solicitar un nuevo enlace.

Recuperación y reenvío no informan si el email tiene cuenta. Su 200 confirma la recepción de la solicitud, no la entrega del correo. El registro conserva el conflicto 409 por email duplicado.

La API no incorpora Swagger/OpenAPI. Los contratos se describen en api.ts y en esta documentación; pueden probarse con una herramienta HTTP respetando JWT, CORS y el entorno aislado.

Fuente: backend/src/auth.ts, auth-postgres.ts y cuenta-correo.ts.

9. API privada de consultorio

Todas estas rutas requieren Authorization: Bearer y usan el usuarioId validado. Las rutas son relativas a /api.

Método y ruta	Función
GET /pacientes	Lista de fichas de la cuenta.
GET /pacientes/siguiente-codigo	Sugerencia sin reserva de código.
GET /pacientes/pendientes-agendar	Activos sin sesión futura/en curso y su última no cancelada.
GET /pacientes/:codigo	Ficha individual.
GET /pacientes/:codigo/proximas	Reservas no terminadas ni canceladas.
GET /pacientes/:codigo/sesiones	Todas las Realizada, por fecha/hora/id descendentes.
POST /pacientes	Alta independiente o conjunta con reserva.
PUT /pacientes/:codigo	Edición de ficha; creación e inicio del tratamiento protegidos.
POST /pacientes/:codigo/baja	Baja lógica y cancelación de reservas aún no iniciadas.
GET /calendario	Franjas y sesiones; fecha semanal y excluirSesion opcionales.
GET /sesiones/agendadas	Programada futuras/en curso; filtro codigoPaciente opcional.
GET /sesiones/ultimas	Una última Realizada por paciente de la cuenta.
GET /sesiones/pendientes	Realizada con saldo positivo.
GET /sesiones/:id	Detalle de sesión propia.
POST /sesiones	Reserva y Nota opcional. Devuelve 201.
PUT /sesiones/:id	Edición conjunta; valida paciente, intervalo, importes y textos.
POST /sesiones/:id/realizar	Confirmación manual de asistencia.
POST /sesiones/:id/cancelar	Desagendar Programada futura.
GET /dashboard	Cuatro cantidades base del resumen.

GET /sesiones/semana, GET /agenda y GET /horarios/:fecha/:horario permanecen por compatibilidad. Los antiguos GET /horarios/disponibles, GET /horarios/disponibles/todos y POST /horarios responden **410** en PostgreSQL: la disponibilidad actual se consulta en calendario. No existe DELETE de fichas o sesiones.

Fuente: backend/src/api.ts y db/consultorio-postgres.ts.

10. Contratos JSON, fechas y errores HTTP

Ejemplo de reserva. La fecha debe reemplazarse por una habilitada al ejecutar:

```
{
  "codigoPaciente": "P-001",
  "fecha": "2026-11-03",
  "horario": "15:30",
  "duracionMinutos": 40,
  "modalidad": "Virtual",
  "importeCentavos": 3000000,
  "postIt": "Preparar el próximo encuentro."
}
```

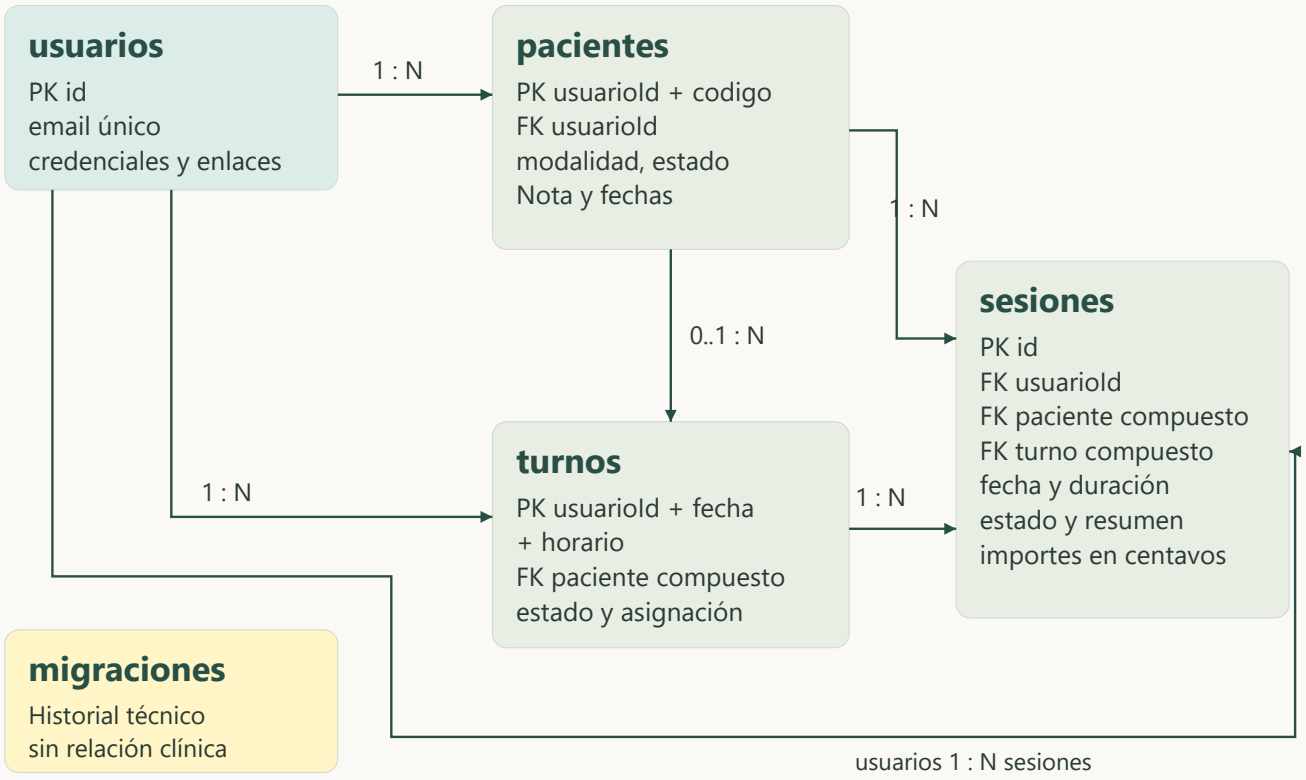
La respuesta de sesión incluye id numérico, fecha, horario, horarioFin, duracionMinutos, codigoPaciente, modalidad, estado, importeCentavos, pagadoCentavos, pendienteCentavos y resumen. Los listados agendados agregan esProxima y postIt; ese postIt es una proyección de la ficha, no una columna de sesiones.

GET /dashboard conserva pacientesActivos, sesionesSemana, pendientesCobro y pendientesAgendar. Inicio combina esa respuesta con agendadas y últimas para mostrar sus seis contadores. No muestra actualmente sesionesSemana como tarjeta.

Estado	Interpretación
200 / 201	Consulta o actualización / creación confirmada.
400	Campos, tipos, longitudes, fechas o reglas inválidas.
401 / 403	Sin autenticación válida / origen o email pendiente rechazado.
404 / 409	Recurso inexistente o ajeno / conflicto de código, reserva o estado.
410 / 413 / 429	Operación antigua retirada / JSON mayor de 32 KB / límite de solicitudes.
500 / 503	Fallo interno genérico / servicio o base no disponible.

El error habitual es { mensaje }. Las fechas de calendario usan YYYY-MM-DD y las horas HH:mm, sin convertirlas en instantes UTC. La creación de cuenta usa ISO UTC. JSON conserva números para IDs, contadores y centavos.

11. Modelo entidad-relación



El profesional se representa en usuarios. Cada fila de pacientes, turnos y sesiones tiene usuarioId y referencia al mismo propietario. El código de paciente por sí solo no es una clave global.

Una sesión tiene un paciente y un turno existentes de su mismo profesional. La relación con turnos utiliza usuarioId, fecha y horario. Un inicio puede conservar varias sesiones Cancelada y, como máximo, una no cancelada, por el índice único parcial.

turnos mantiene el inicio reservado o liberado y referencias heredadas del modelo anterior. No representa hoy una tabla de rangos laborables ni define las franjas disponibles del calendario.

migraciones es independiente de los datos clínicos. Registra versión, archivo, checksum y momento de aplicación; no referencia pacientes ni profesionales.

Las claves foráneas compuestas previenen asociaciones entre cuentas. No filtran por sí mismas las consultas SELECT ni verifican todos los campos duplicados de turno/sesión. Esas operaciones se coordinan en la transacción de negocio. No hay borrado en cascada, triggers clínicos ni Row-Level Security.

Fuente: backend/migrations/001_esquema_inicial.sql y 003_calendario_sesiones.sql.

12. Diccionario de datos: usuarios

Columna	Tipo PostgreSQL	Restricción o propósito
id	INTEGER IDENTITY	PK, generado por defecto.
email	TEXT NOT NULL	Índice único sobre lower(email).
passwordHash	TEXT NOT NULL	Argon2id codificado, nunca devuelto por la API.
fechaCreacion	TIMESTAMPTZ(3) NOT NULL	Instante de creación asignado en el servidor.
emailConfirmado	BOOLEAN NOT NULL	Predeterminado false para altas nuevas.
versionCredenciales	INTEGER NOT NULL	Predeterminado 0, CHECK mayor o igual que cero.
confirmacionHash	TEXT NULL UNIQUE	SHA-256 hexadecimal de 64 caracteres.
confirmacionVence	TIMESTAMPTZ NULL	Vencimiento del enlace de confirmación.
recuperacionHash	TEXT NULL UNIQUE	SHA-256 del enlace de recuperación.
recuperacionVence	TIMESTAMPTZ NULL	Vencimiento del enlace de recuperación.

Cada pareja hash/vencimiento debe estar completamente nula o contener ambos valores válidos. No se guardan tokens de correo en claro. No hay una tabla de JWT ni un padrón de pacientes en esta entidad.

La migración 008 habilita por compatibilidad las cuentas anteriores a la aplicación de 007, identificadas por fechaCreacion frente al historial. No altera el DEFAULT false para las nuevas.

Autenticación de contraseña

argon2.hash utiliza explícitamente Argon2id, sal aleatoria y parámetros de costo predeterminados de la biblioteca. El resultado incluye algoritmo, sal y parámetros. argon2.verify comprueba la candidata exacta. No existe una clave para descifrar la contraseña original.

Para un email inexistente también se verifica un hash auxiliar en memoria, evitando una salida inmediata sin hashing. Esto no constituye una auditoría formal contra todos los canales de tiempo.

Fuente: migraciones 001, 007 y 008; backend/src/auth.ts y auth-postgres.ts.

13. Diccionario de datos: pacientes

Columna	Tipo PostgreSQL	Regla actual
usuarioId	INTEGER NOT NULL	PK compuesta y FK a usuarios(id).
codigo	TEXT COLLATE C NOT NULL	PK compuesta; formato de 1 a 12 ASCII y «nuevo» reservado.
modalidad	TEXT NOT NULL	CHECK Presencial o Virtual.
estado	TEXT NOT NULL	CHECK longitud 1 a 40. La API limita estados y baja explícita.
proximaSesion	TEXT NULL	Etiqueta de hasta 30 caracteres; la respuesta se deriva de sesiones vigentes.
frecuencia	TEXT NOT NULL	Campo histórico de 1 a 40 caracteres; alta actual A confirmar, sin recurrencia.
diaHabitual	TEXT NULL	CHECK con los siete nombres de días.
horarioHabitual	TIME WITHOUT TIME ZONE NULL	Sin segundos y menor que 24:00. Informativo en la ficha.
fechaCreacion	DATE NOT NULL	Asignada e inmutable por la API.
fechaInicioTratamiento	DATE NULL	MIN(fecha) de Realizada; nula si no existen.
motivoConsulta	TEXT NOT NULL	Cadena vacía permitida; hasta 3000 caracteres.
postIt	TEXT NOT NULL	Nota vigente, vacía permitida; hasta 300 caracteres.

PK: (usuarioId, codigo). Un índice único adicional (usuarioId, lower(codigo)) evita códigos que difieran solo en mayúsculas. La colación C mantiene la comparación exacta de las referencias.

La API admite actualizaciones de ficha y protege ambas fechas. Renombrar actualiza referencias de turnos y sesiones dentro de la misma transacción. Las dos FK de paciente son diferibles para confirmar el cambio conjunto.

No se agregan nombre real, DNI, domicilio ni teléfono. La identificación por código reduce información identificatoria necesaria, pero no equivale a anonimización completa del contenido libre.

Fuente: migraciones 001 y 006; validar-paciente.ts y db/consultorio-postgres.ts.

14. Diccionario de datos: turnos

Columna	Tipo PostgreSQL	Restricción
usuarioId	INTEGER NOT NULL	Parte de PK, FK a usuarios(id).
fecha	DATE NOT NULL	Parte de PK. Rango 0001-01-01 a 9999-12-31.
horario	TIME WITHOUT TIME ZONE NOT NULL	Parte de PK; segundos cero, menor que 24:00.
codigoPaciente	TEXT COLLATE C NULL	FK compuesta al paciente del mismo profesional.
modalidad	TEXT NULL	Presencial o Virtual cuando está asignado.
estado	TEXT NOT NULL	Disponible, Programado o Liberado.

PK: (usuarioId, fecha, horario). El CHECK de asignación exige paciente y modalidad para Programado, y ambos NULL para Disponible o Liberado.

El calendario actual no ofrece filas libres de esta tabla como catálogo manual. Calcula intervalos a partir de sesiones. Al reservar, ocuparTurno utiliza INSERT ... ON CONFLICT para crear o reutilizar el inicio; la sesión conserva la FK compuesta a ese turno.

Al reprogramar o cancelar, liberarTurno solo lo marca Liberado cuando no queda otra sesión no cancelada ocupando ese mismo inicio. El registro del turno puede permanecer como referencia de sesiones canceladas.

Tabla técnica migraciones

Columna	Tipo	Restricción
version	INTEGER	PK, mayor que cero.
archivo	TEXT NOT NULL	Único.
checksum	TEXT NOT NULL	SHA-256 hexadecimal de 64 caracteres.
aplicadaEn	TIMESTAMPTZ NOT NULL	DEFAULT CURRENT_TIMESTAMP.

El rol de la aplicación no accede al historial de migraciones. Su propietario y las herramientas administran estructura e historial.

Fuente: migración 001; scripts/migrar.mjs; db/consultorio-postgres.ts.

15. Diccionario de datos: sesiones

Columna	Tipo PostgreSQL	Restricción o salida
id	INTEGER IDENTITY	PK, identificador numérico de 32 bits.
usuarioId	INTEGER NOT NULL	FK al profesional.
fecha	DATE NOT NULL	Rango de fechas válidas del proyecto.
horario	TIME WITHOUT TIME ZONE NOT NULL	Segundos cero, menor que 24:00.
codigoPaciente	TEXT COLLATE C NOT NULL	FK (usuarioId, codigoPaciente) a pacientes.
modalidad	TEXT NOT NULL	Presencial o Virtual.
estado	TEXT NOT NULL	Programada, Realizada o Cancelada; DEFAULT Programada.
importeCentavos	INTEGER NOT NULL	DEFAULT 0; entre 0 y 999999999.
pagadoCentavos	INTEGER NOT NULL	DEFAULT 0; entre 0 y el importe.
resumen	TEXT NOT NULL	DEFAULT vacío; máximo 100 caracteres.
duracionMinutos	INTEGER NOT NULL	DEFAULT 60; CHECK 30, 40, 50 o 60.

FK de turno: (usuarioId, fecha, horario). El índice sesiones_horario_unico es único sobre esas columnas **solo para estado distinto de Cancelada**. sesiones_paciente indexa (usuarioId, codigoPaciente).

Valores derivados

pendienteCentavos se calcula como importeCentavos - pagadoCentavos. horarioFin se calcula desde fecha, horario y duración. esProxima y postIt se agregan al consultar reservas, sin persistirse en sesiones.

Los índices únicos de inicio no detectan una superposición entre inicios diferentes. Por ejemplo 15:30-16:10 y 16:00-16:40 requieren la validación de intervalos del backend bajo bloqueo. No existe una restricción EXCLUDE de rangos.

Los huecos de IDs tras un rollback son normales y no representan una sesión faltante. El ID identifica una fila, no una numeración continua de encuentros.

Fuente: migraciones 001 a 005 y db/consultorio-postgres.ts.

16. Integridad, tipos e índices

Las seis claves foráneas conectan usuarios, pacientes, turnos y sesiones. Las compuestas incluyen usuarioId y evitan referencias a pacientes o turnos de otra cuenta. Se usan NOT NULL, CHECK, PK y UNIQUE como segunda defensa además de los validadores de Express.

Garantía	Implementación
Email único	Índice lower(email).
Código único por cuenta	PK exacta e índice (usuarioId, lower(codigo)).
Inicio no cancelado único	Índice parcial de sesiones por propietario/fecha/hora.
Referencias válidas	FK simples y compuestas, sin borrado en cascada.
Cobro consistente	CHECK 0 ≤ pagadoCentavos ≤ importeCentavos.
Textos acotados	CHECK de motivo, Nota y resumen.
Estado y duración	CHECK de modalidades, estados y cuatro duraciones permitidas.
Cambio de código atómico	FK de paciente diferibles, comprobadas al COMMIT.

Fechas y dinero

DATE representa días de calendario y TIME WITHOUT TIME ZONE representa la hora local del consultorio. SQL usa to_char para devolver YYYY-MM-DD y HH:mm, sin desplazarlos por la zona del proceso. TIMESTAMPTZ conserva instantes de cuenta y enlaces de correo. calendario.ts obtiene el reloj de Buenos Aires mediante Intl.DateTimeFormat.

Los importes se almacenan como INTEGER en centavos. Por ejemplo, ARS 30.000 corresponde a 3000000. Esto evita usar decimales binarios para saldos. Los formularios convierten entre texto en pesos y enteros; SQL devuelve contadores con ::integer para conservar números JSON.

Una restricción de fila no sustituye todas las reglas temporales o las consultas filtradas. El calendario, la ventana de fechas y las reservas contiguas son validaciones de aplicación.

Referencia conceptual: PostgreSQL 18, Constraints. https://www.postgresql.org/docs/18/ddl-constraints.html

17. Migraciones y administración del esquema

scripts/migrar.mjs ordena archivos por versión, verifica sus nombres y SHA-256 y comprueba el historial aplicado. Normaliza BOM y CRLF para calcular checksums reproducibles. Un advisory lock serializa migradores de una misma base.

Cada migración pendiente se ejecuta en una transacción junto con su registro. Un fallo revierte esa versión, sin deshacer versiones anteriores ya confirmadas. No hay migración automática al arrancar Express ni comando de reversión automática de versiones confirmadas.

Versión	Efecto actual
001	Cuatro tablas funcionales, claves, restricciones, índices y permisos de aplicación.
002	Resumen opcional de sesión, máximo 100 caracteres.
003	Duración, Cancelada e índice único parcial por inicio.
004	Completa con 60 minutos las duraciones antiguas nulas y exige NOT NULL.
005	Duraciones permitidas 30/40/50/60, sin reescribir sesiones.
006	Inicio de tratamiento nulo o derivado de la primera Realizada.
007	Confirmación, recuperación y versión de credenciales en usuarios.
008	Excepción de acceso para cuentas anteriores a 007.

Desde backend, con credenciales privadas del destino correspondiente:

```
npm.cmd run db:status
npm.cmd run db:migrate
npm.cmd run db:status:test
npm.cmd run db:migrate:test
```

psique_migrador administra psique. psique_app solo posee SELECT, INSERT y UPDATE de las cuatro tablas y USAGE de dos secuencias; no DELETE, TRUNCATE, DDL ni acceso a migraciones. psique_test es propietario de su base aislada. No se usa postgres para ejecutar la API.

Nunca editar un archivo ya aplicado: agregar una nueva versión. Antes de cambios en desarrollo corresponde un respaldo con pg_dump y verificación de restauración; los respaldos locales existentes no constituyen por sí solos una política de recuperación productiva.

18. Transacciones y control de concurrencia

Cada escritura de consultorio ejecuta estos pasos sobre el mismo cliente:

1. pool.connect y BEGIN.
2. SET LOCAL lock_timeout de 3 s e idle_in_transaction_session_timeout de 10 s.
3. SELECT id FROM usuarios WHERE id = $1 FOR UPDATE con el propietario del JWT.
4. Lecturas, validaciones y escrituras de la operación.
5. COMMIT antes de devolver el resultado exitoso.
6. Ante fallo, ROLLBACK; siempre release en finally. Una conexión dañada se descarta.

El bloqueo serializa cambios de una misma cuenta. Una segunda reserva espera y valida el estado ya confirmado por la primera. Profesionales diferentes bloquean filas distintas. Es una solución sencilla, aunque limita el paralelismo de escrituras dentro de una cuenta.

Operaciones atómicas relevantes

- Alta de paciente y primera reserva.
- Reserva y cambio de Nota vigente.
- Reprogramación, ocupación del destino, liberación del origen y etiquetas.
- Renombrado de paciente y referencias en turnos/sesiones.
- Confirmación o corrección de asistencia e inicio del tratamiento derivado.
- Baja lógica y cancelación de reservas aún no iniciadas.

Solo las FK de paciente se difieren al renombrar. Si una referencia sigue inválida, COMMIT falla y no se informa éxito.

SQLSTATE 23505 de restricciones conocidas se traduce a 409. Espera de bloqueo agotada 55P03 y deadlock 40P01 se traducen a indisponibilidad, después de revertir. Errores inesperados reciben mensajes genéricos.

No se reintentan escrituras automáticamente. Si se pierde la respuesta de COMMIT, el cliente puede desconocer si se confirmó; debe consultar el estado antes de repetir. La persistencia no elimina esa ambigüedad de red.

Fuente: db/consultorio-postgres.ts. Referencia: https://node-postgres.com/features/transactions

19. Autorización, JWT y seguridad HTTP

JWT usa HS256 con sub, iat, exp, iss=psique, aud=psique-app y versionCredenciales. Dura 3600 segundos. El middleware verifica firma, algoritmo permitido, claims, antigüedad y sujeto; después consulta que el usuario exista, esté confirmado y mantenga la versión del token.

Solo entonces asigna res.locals.usuarioId. Las rutas no aceptan un propietario alternativo desde el cuerpo o la URL. Cada consulta privada, JOIN, sugerencia y escritura filtra por ese ID. Un recurso ajeno responde 404. Las FK compuestas previenen referencias cruzadas, pero no sustituyen esos filtros.

Medida	Alcance implementado
SQL parametrizado	Valores separados del SQL mediante $1, $2 y arreglos.
Validación	Campos permitidos, tipos, longitudes, fechas reales, importes y relaciones.
CORS exacto	Origen web y, opcionalmente, https://localhost para Android. GET/POST/PUT, sin comodines ni cookies.
JSON acotado	32 KB. Errores sin consultas, claves ni cuerpos del proveedor.
Helmet	Cabeceras de seguridad para API JSON; HSTS solo en producción.
Caché	no-store, sin ETag ni X-Powered-By.
Límites de acceso	Login 30/15 min y registro 10/hora por IP; correo con límites específicos.
Proxy	Confianza deshabilitada salvo redes explícitas verificadas.

CORS regula JavaScript del navegador; una herramienta sin Origin puede llegar a la API y aun así necesita JWT en rutas privadas. No es un mecanismo de autenticación.

Los cupos están en memoria de una instancia, se pierden al reiniciar y no se comparten entre réplicas. No se implementaron RLS, auditoría clínica, refresh tokens, bloqueo por inactividad ni cifrado adicional de contenidos clínicos.

Fuente: auth.ts, auth-postgres.ts, configuracion.ts, seguridad-http.ts y reglas-api.ts.

20. Integración de Resend y enlaces de cuenta

correo.ts define una interfaz Correo y un adaptador Resend activado por variables privadas. Utiliza fetch HTTPS con timeout de 8 segundos, sin SDK adicional. Los errores expuestos son genéricos y no incluyen claves, cuerpos del proveedor ni enlaces.

Confirmación y recuperación

- Se generan 32 bytes criptográficos aleatorios, codificados base64url.
- La base guarda solo su hash SHA-256 y vencimiento.
- Confirmación vence a las 24 horas y recuperación a los 30 minutos.
- Entre envíos por cuenta se exige un minuto. Un enlace nuevo reemplaza al anterior del mismo tipo.
- El destino del enlace proviene de FRONTEND_ORIGIN, no de Host ni de un parámetro del cliente.
- El token viaja en el fragmento de la URL, que no se envía al servidor como consulta HTTP.
- La pantalla lo consume mediante una acción explícita, no por abrirla.

Un UPDATE condicionado confirma el email o consume el enlace de recuperación de forma atómica. Restablecer guarda el nuevo Argon2id, limpia el enlace e incrementa versionCredenciales en la misma sentencia. Los JWT anteriores dejan de autorizar solicitudes.

Las solicitudes públicas responden de forma genérica sin esperar al proveedor. No hay cola persistente: una caída abrupta puede interrumpir un envío. El registro sí espera el resultado para informar correoEnviado. El cierre ordenado espera envíos en curso.

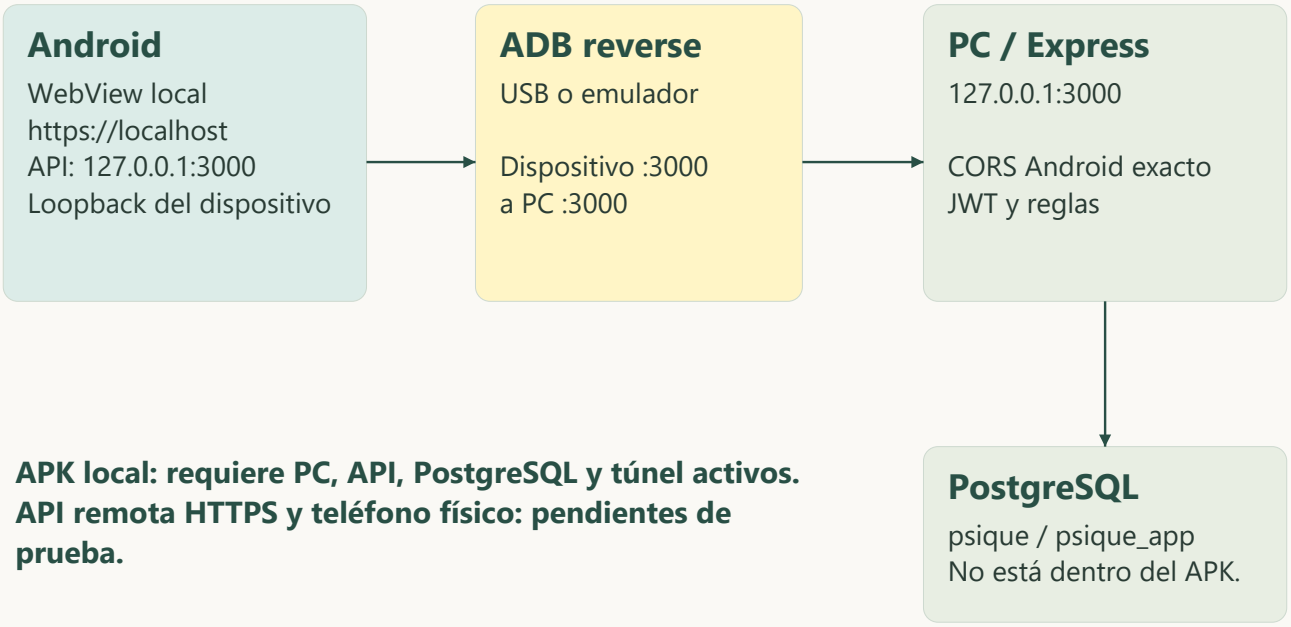
Estado operativo y pruebas

La configuración local existente usa Resend sin dominio propio y limita el destinatario al autorizado. El adaptador no redirige enlaces de otra cuenta a esa casilla. El envío general requiere dominio verificado. Durante esta entrega se usó un buzón ficticio aislado y no se enviaron correos reales.

La recepción real de correo se verificó en una etapa anterior y está registrada en README/POSTGRESQL.md; no se contabiliza como un envío nuevo de esta entrega.

Referencia del proveedor: https://resend.com/docs/knowledge-base/403-error-resend-dev-domain

21. Android con Capacitor y conectividad



capacitor.config.ts define **com.francocaudeli.psique**, nombre PSIQUE y webDir **www/browser**. Los archivos Angular se copian al proyecto nativo y se cargan desde https://localhost. No hay server.url, live reload ni carga de un sitio remoto como interfaz.

Qué incorpora Android

El contenedor WebView reutiliza pantallas, formularios, servicios y JWT existentes. El único plugin adicional es App, para el botón Atrás de Ionic. La interfaz contempla teclado y márgenes de las barras del sistema. El backend y sus secretos permanecen fuera del APK.

127.0.0.1 dentro del teléfono o emulador es su propio loopback. **adb reverse tcp:3000 tcp:3000** permite que el puerto 3000 del dispositivo llegue al puerto 3000 de esta PC. Requiere dispositivo autorizado, PC y API activos. 10.0.2.2 es una alternativa específica del emulador y no se utiliza en el bundle actual.

El modo debug permite HTTP únicamente hacia 127.0.0.1 mediante su política de red y activa contenido mixto solo bajo BuildConfig.DEBUG. La configuración general y release rechazan HTTP. Una tarea Gradle impide generar release con el bundle local.

Una API HTTPS remota permitirá independencia de esta PC cuando exista y se pruebe. No hay hosting, App Links ni publicación en Google Play. Los enlaces de correo siguen abriéndose en la versión web.

Fuentes: capacitor.config.ts, android/, scripts/preparar-android.mjs. Referencia: https://capacitorjs.com/docs/android

22. Variables de entorno y secretos

Variable	Propósito y condición
DATABASE_URL	Conexión privada habitual: psique_app en psique. Obligatoria.
MIGRATION_DATABASE_URL	Solo migrador: psique_migrador en psique.
APP_DATABASE_URL	Verificación de permisos de psique_app, no conexión del servidor habitual.
TEST_DATABASE_URL	Solo pruebas: psique_test en psique_test.
JWT_SECRET	Secreto aleatorio de al menos 32 bytes. Sin valor por defecto.
NODE_ENV	development, test o production. Predeterminado development.
HOST / PORT	127.0.0.1:3000 local. Producción admite 0.0.0.0 y exige PORT.
FRONTEND_ORIGIN	Origen web exacto. Local http://127.0.0.1:4200; HTTPS en producción.
CAPACITOR_ANDROID_ORIGIN	Opcional y exclusivamente https://localhost.
DATABASE_SSL_MODE	disable local; verify-full obligatorio en producción.
DATABASE_SSL_CA_FILE	Certificado CA público confiable opcional para verify-full.
TRUSTED_PROXY_CIDRS	Redes/IP verificadas del proxy. Vacío no confía en encabezados de IP.
CORREO_PROVEEDOR	resend cuando el envío está configurado.
CORREO_REMITENTE	Remitente autorizado por el proveedor.
RESEND_API_KEY	Secreto privado del backend para enviar correos.
CORREO_DESTINATARIO_PRUEBA	Restringe envíos a una casilla autorizada durante pruebas.
PSIQUE_API_URL	URL pública HTTPS de compilación web/Android remota, terminada en /api.

Las cuatro variables de conexión seleccionan roles y bases distintos. El entorno de pruebas rechaza credenciales de desarrollo presentes. Las URL PostgreSQL no admiten parámetros que sustituyan su destino o política TLS.

No se incluyen valores reales en esta entrega. .env y el almacén DPAPI local no se distribuyen. En la PC del autor, un lanzador privado suministra credenciales solo al proceso; en otra instalación deben crearse secretos propios. Una URL pública de API no equivale a una connection string PostgreSQL.

Fuente: configuracion.ts, db/postgres.ts, db/seguridad-postgres.ts, correo.ts y ejemplos .env del backend.

23. Instalación nueva en Windows y PostgreSQL

Requisitos: Node 24.x desde 24.14.1, npm, PostgreSQL 18.6 y permisos para preparar bases locales. Android Studio, JDK 21 y SDK 36 solo son necesarios para compilar Android. No se requiere Docker.

Preparar una instalación vacía

Los comandos siguientes son una guía para **otro entorno nuevo**, ejecutada como administrador PostgreSQL. No deben repetirse indiscriminadamente sobre bases existentes.

```
CREATE ROLE psique_migrador LOGIN NOSUPERUSER NOCREATEDB
  NOCREATEROLE NOREPLICATION NOBYPASSRLS;
CREATE ROLE psique_app LOGIN NOSUPERUSER NOCREATEDB
  NOCREATEROLE NOREPLICATION NOBYPASSRLS;
CREATE ROLE psique_test LOGIN NOSUPERUSER NOCREATEDB
  NOCREATEROLE NOREPLICATION NOBYPASSRLS;
CREATE DATABASE psique OWNER psique_migrador ENCODING 'UTF8';
CREATE DATABASE psique_test OWNER psique_test ENCODING 'UTF8';
REVOKE ALL ON DATABASE psique FROM PUBLIC;
REVOKE ALL ON DATABASE psique_test FROM PUBLIC;
GRANT CONNECT ON DATABASE psique TO psique_app;
```

En psql, asignar contraseñas propias con **\password psique_migrador**, **\password psique_app** y **\password psique_test**, que solicitan entrada oculta. No escribirlas en scripts de entrega.

En psql, configurar los esquemas de esas bases nuevas como administrador o propietario:

```
\connect psique
REVOKE CREATE ON SCHEMA public FROM PUBLIC;
GRANT USAGE ON SCHEMA public TO psique_app;
\connect psique_test
REVOKE CREATE ON SCHEMA public FROM PUBLIC;
```

No conceder a pruebas acceso a desarrollo. Cada propietario conserva sus permisos.

Suministrar MIGRATION_DATABASE_URL privadamente y ejecutar db:migrate desde backend. Luego, en un entorno separado que contenga solo TEST_DATABASE_URL, ejecutar db:migrate:test. Los archivos SQL ya definen las tablas y permisos; no recrearlos manualmente.

Esta entrega no ejecutó instalación, creación de roles o migraciones sobre desarrollo. El historial existente se conservó. Para esquemas ya preparados, usar db:status antes de aplicar cambios y generar un respaldo previo cuando corresponda.

24. Ejecutar web, API y pruebas localmente

Dependencias y configuración

Desde la raíz, instalar npm.cmd ci. En backend, ejecutar otro npm.cmd ci. No es necesario instalarlos globalmente. Copiar .env.example a .env únicamente si no existe, completar JWT_SECRET de forma privada y configurar el correo para habilitar nuevas cuentas.

En una PC nueva se necesita una cuenta propia de Resend y una API key con permiso de envío. Sin dominio verificado se usa el remitente de prueba y el email autorizado de esa cuenta como único destinatario. Configurar las cuatro variables de correo de la sección anterior antes del primer registro; no reutilizar las credenciales del autor.

En otra PC puede suministrarse DATABASE_URL mediante un archivo privado externo o un entorno de proceso protegido. La conexión reúne protocolo PostgreSQL, rol, contraseña codificada, servidor, puerto y base. Completarla únicamente en el entorno privado, sin imprimirla ni guardarla en código.

Ejecución en la PC del autor

```
# Terminal 1. El lanzador privado se ubica en backend.
& "$env:LOCALAPPDATA\PSIQUE\postgresql-local\Herramientas-PostgreSQL.ps1" -Tarea dev
# Terminal 2. Desde la raíz de APP-PSIQUE.
npm.cmd start
```

Abrir http://127.0.0.1:4200. API: http://127.0.0.1:3000/api. PostgreSQL debe estar iniciado. Ctrl+C detiene cada proceso. No iniciar dos API en el mismo puerto.

Ejecución portable sin el lanzador privado

Desde backend, con el entorno privado ya configurado, npm.cmd run dev inicia el soporte --watch. Alternativamente, Node permite cargar un archivo propio externo:

```
node --env-file=C:\Privado\psique-app.env --watch src/server.ts
# Para ejecutar JavaScript compilado, desde backend:
npm.cmd run build
node --env-file=C:\Privado\psique-app.env dist/server.js
```

Con DATABASE_URL suministrada por el entorno, npm.cmd start ejecuta el mismo servidor compilado. Desde la raíz, npm.cmd run build genera www/browser para uso local. build:production exige una PSIQUE_API_URL HTTPS real y no publica por sí mismo.

El archivo privado de pruebas debe contener solo TEST_DATABASE_URL, NODE_ENV=test y su política TLS local, sin conexiones ni claves reales de desarrollo. Ejecutar las pruebas con ese entorno, nunca sobre psique.

25. Compilar, instalar y ejecutar Android

Android Studio abre android/. El proyecto usa SDK 36, Build-Tools que resuelve Gradle, Platform-Tools y JDK 21. En esta PC Studio incorpora JBR 25, incompatible con Gradle 8.14.3: se utiliza Temurin 21.0.13+9 portátil sin cambiar Java del sistema.

Desde la raíz, los comandos normales son:

```
npm.cmd run android:sync:local
npm.cmd run android:open
npm.cmd run android:apk:local
```

El último compila Angular optimizado sin source maps, sincroniza Capacitor y ejecuta assembleDebug. Resultado: **android/app/build/outputs/apk/debug/app-debug.apk**. Para actualizarlo, repetir sincronización y compilación; npm start no cambia un APK instalado.

En el backend local habilitar CAPACITOR_ANDROID_ORIGIN=https://localhost al iniciarlo, conservando FRONTEND_ORIGIN web. Con el emulador encendido o un dispositivo USB autorizado:

```
$adb = "$env:LOCALAPPDATA\Android\Sdk\platform-tools\adb.exe"
& $adb devices
& $adb reverse tcp:3000 tcp:3000
& $adb install -r .\android\app\build\outputs\apk\debug\app-debug.apk
& $adb shell am start -n com.francocaudeli.psique/.MainActivity
```

Con varios dispositivos, agregar -s IDENTIFICADOR. Al reconectar puede ser necesario repetir reverse. Para correos con enlaces web locales, iniciar Angular 4200 y crear también reverse de ese puerto. No se implementaron App Links.

El APK final comprobado tiene **4.459.210 bytes** y SHA-256 **946453476033A8342A02FF3185795CCC7870DEC055866779C3288CBC69663ED2**. apksigner confirmó firma v2. Se instaló en Medium_Phone_API_37.0. El build es debug, sin publicación ni firma de distribución.

Con una futura API HTTPS, android:sync y android:apk requieren PSIQUE_API_URL explícita. No habilitan una API remota ni cambian la base. ANDROID.md contiene las instrucciones operativas ampliadas.

26. Pruebas ejecutadas y evidencia

Comprobación repetida para esta entrega	Resultado
npm run test:config, raíz	37 pruebas aprobadas, 0 fallos, 0 omitidas.
test:postgres, lanzador privado	108 pruebas aprobadas en psique_test, 0 fallos, 0 omitidas.
npm test, backend	87 pruebas aprobadas, 0 fallos, 0 omitidas. Incluye compilación TypeScript.
npm run build, raíz	Compilación web correcta.
apksigner verify --verbose	APK existente verificado, firma v2 válida.
Capturas Android	Login y recorridos privados reales con API aislada, datos ficticios y buzón simulado.

Los resultados de etapas anteriores no se suman a estas cantidades. La suite histórica usa SQLite temporal y no certifica que todas sus reglas antiguas sean el calendario actual. La suite PostgreSQL es la referencia del backend habitual.

Para repetirlas: test:config se ejecuta en la raíz; npm test y test:postgres, en backend. Con el entorno de pruebas en un archivo privado externo, después de compilar backend:

```
node --env-file=C:\Privado\psique-test.env --test `
  --test-concurrency=1 test-postgres/*.test.mjs
```

Cobertura representativa

Registro/login/JWT, confirmación y recuperación, filtros por propietario, tablas y migraciones, reservas y cruces, duración, baja y cancelación, fechas derivadas, notas, cobros, últimas sesiones, histórico, rollback, concurrencia, COMMIT, indisponibilidad y persistencia disponen de casos automatizados actuales.

La etapa Android anterior verificó además teclado real, botón Atrás, pantalla pequeña, logout, cuenta B sin datos ajenos y simulación de red caída. Se conserva su evidencia operativa en ANDROID.md. La generación completa del APK no se repitió para redactar documentos; su hash y firma corresponden al mismo archivo probado.

Evidencias incluidas

evidencias/pruebas-frontend.txt, pruebas-postgresql.txt, pruebas-backend-historico.txt, compilacion-web.txt y firma-apk.txt contienen las salidas. capturas-integracion.json documenta operaciones de preparación, entorno aislado y retirada de la cuenta ficticia.

No se enviaron correos reales ni se detuvo PostgreSQL de desarrollo. Un test aprobado acredita ese caso y entorno; no reemplaza una auditoría de seguridad ni una prueba en teléfono físico.

27. Despliegue previsto y resguardos pendientes

La arquitectura preparada admite un frontend estático y una API remota con PostgreSQL administrado. **No existe despliegue comprobado en Render, Neon ni otro proveedor.** No se crearon dominios, infraestructura ni servicios remotos para esta entrega.

Preparación ya implementada

- Destino Angular por compilación mediante PSIQUE_API_URL HTTPS, sin credenciales ni loopback de producción.
- HOST/PORT y origen exacto validados por entorno.
- Política PostgreSQL verify-full en producción, con certificado y nombre válidos y TLS 1.2 mínimo.
- Health checks, cierre ordenado y límites del pool.
- Separación de credenciales de aplicación, migrador y pruebas.

Trabajo previo a una operación remota

Verificar roles/nombres admitidos por la instancia, conectividad SQL, CA confiable, proxy y sus redes, HTTPS público, cabeceras del sitio Angular, reescritura SPA, dominio remitente, capacidad, backups y restauración. Los nombres psique/psique_app están exigidos en el código actual y deben comprobarse en cualquier proveedor.

Información clínica real

Esta versión académica no acredita una preparación integral para producción clínica. Faltan una evaluación formal de privacidad y obligaciones aplicables, política de retención y borrado, auditoría de accesos, controles operativos, restauración ensayada y respuesta a incidentes.

El transporte HTTP local por ADB es exclusivo de prueba. Los JWT son firmados, no cifrados. La base no incorpora cifrado adicional de campos clínicos ni cifrado de extremo a extremo. El token en sessionStorage no utiliza un almacén nativo protegido. No se implementaron MFA, bloqueo por inactividad ni revocación individual al logout.

El uso de alias reduce exposición, pero los textos libres pueden contener información sensible. Las capturas y demostraciones deben continuar utilizando datos ficticios. No se afirma cumplimiento legal ni anonimato garantizado.

28. Operación, trazabilidad y referencias

Síntoma	Comprobación o procedimiento
API no inicia	Revisar JWT_SECRET, DATABASE_URL, servicio, rol, esquema y puerto, sin imprimir secretos.
401	Revisar acceso, vencimiento y versión de credenciales; volver a Login.
403	Revisar confirmación de cuenta u origen exacto del cliente.
409 al reservar	Refrescar calendario y comprobar intervalo completo y próxima fecha del paciente.
503	Revisar disponibilidad PostgreSQL o configuración de correo según el mensaje. No cambiar a SQLite.
Correo no recibido	Revisar spam, remitente, destinatario permitido y configuración Resend. Un 200 de solicitud no garantiza entrega.
APK sin API	Comprobar adb devices, reverse, puerto y CORS Android del backend.
Gradle falla con Java 25	Seleccionar JDK 21 del proyecto, sin actualizar AGP/Gradle arbitrariamente.

Trazabilidad de decisiones

Rutas y errores: backend/src/api.ts. Identidad: auth.ts y auth-postgres.ts. Datos y transacciones: db/consultorio-postgres.ts. Agenda: agenda-calendario.ts. Esquema: migrations/001 a 008. Correo: cuenta-correo.ts y correo.ts. Arranque/pool: server.ts y db/postgres.ts. Cliente: src/app/ y environments/. Android: capacitor.config.ts, android/ y preparar-android.mjs.

Se conservaron el código de ejecución y las migraciones durante esta tarea documental. El control de integridad y los resultados de revisión de los PDF se incluyen en evidencias. Las capturas originales y sus recortes están disponibles en recursos/capturas; no se generaron pantallas ficticias.

Referencias oficiales consultadas

- PostgreSQL 18, Constraints: https://www.postgresql.org/docs/18/ddl-constraints.html
- node-postgres, Transactions: https://node-postgres.com/features/transactions
- Capacitor, Android: https://capacitorjs.com/docs/android
- Resend, restricciones del dominio de prueba: https://resend.com/docs/knowledge-base/403-error-resend-dev-domain

Los documentos operativos README.md, backend/POSTGRESQL.md y ANDROID.md amplían comandos e historia de implementación. Sus apartados históricos no reemplazan el estado vigente descrito aquí.